

The Matryoshka Hypencoder

Majd Alkawaas
University of Glasgow
Glasgow, United Kingdom
Majd.Alkawaas@glasgow.ac.uk

Sean MacAvaney
University of Glasgow
Glasgow, United Kingdom
Sean.MacAvaney@glasgow.ac.uk

Abstract

The Hypencoder is a recently-proposed retrieval approach that encodes queries as shallow neural networks (“Q-Nets”) that estimate relevance over pre-computed document embeddings. Inspired by Matryoshka Representation Learning, we show that the Hypencoder can be extended to support multiple sizes of Q-Nets, allowing trade-offs between effectiveness and efficiency when deployed. We find that this “Matryoshka Hypencoder” achieves comparable in-domain effectiveness with approximately $7\times$ fewer active parameters in-domain and half as many active parameters out-of-domain, which corresponds to a $1.6\text{--}3.4\times$ increase in scoring throughput. This work paves the way for practical deployment of Hypencoders.

 <https://github.com/MajdAlkawaas/hypencoder-paper>

CCS Concepts

• Information systems → Information retrieval.

Keywords

Hypencoder, Matryoshka Embeddings, Passage Retrieval

ACM Reference Format:

Majd Alkawaas and Sean MacAvaney. 2026. The Matryoshka Hypencoder. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26)*, July 20–24, 2026, Melbourne, VIC, Australia. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3805712.3809980>

1 Introduction

Neural ranking architectures provide inherent trade-offs in effectiveness, inference cost, and efficiency. For instance, single-vector dense architectures, such as DPR [9], have low storage and inference costs, at the expense of inherent limitations in effectiveness [26]. Other architectures, such as those used by late interaction models [10] and cross-encoders [21], sacrifice inference speed for an increased capability to estimate relevance. Killingback et al. [11] recently proposed the *Hypencoder*, which aims to give single-vector dense models a higher capability to estimate relevance by scoring document vectors using a lightweight query-specific neural network (a *Q-Net*), rather than typical measures of vector similarity (e.g., cosine, inner product). The *Q-Net* (generated by a separate neural model, i.e., a *HyperNetwork* [6]) has greater expressive capacity than traditional similarity measures, and consequently provides higher retrieval effectiveness. The Hypencoder manages

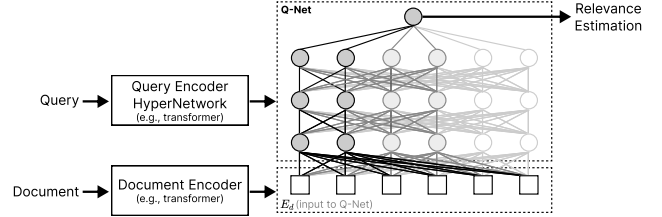


Figure 1: Overview of the Matryoshka Hypencoder architecture. Unlike a typical Hypencoder network, the Matryoshka Hypencoder produces Q-Nets that can run at different sizes, which are represented by shades of gray.

storage costs by leveraging single-vector document encodings, and inference efficiency is managed by using a shallow multi-layer perceptron as the *Q-Net* architecture.

Meanwhile, several works have explored alternative ways to balance trade-offs in single-vector dense models. These efforts usually assume a fixed model, instead focusing on how to efficiently index and search for a given model [5, 12, 16, 18]. A notable exception is Kusupati et al. [13], which proposed learning a single model capable of representing query and document vectors at multiple sizes. These *Matryoshka* models allow a single model to be deployed at different operational points—a greater number of dimensions generally improves effectiveness at the expense of greater storage and inference costs. We refer to this design as the *Matryoshka Principle*: representations are constructed as nested prefixes, such that any prefix constitutes a valid, lower-cost representation, while longer prefixes strictly increase capacity and effectiveness at the expense of additional computation or storage. The *Matryoshka Principle* has also been applied *inside* encoder transformer networks [14, 25], allowing efficiency-effectiveness trade-offs to be made during the process of encoding queries and documents. In this case, a greater number of activated parameters (here, transformer layers) results in higher effectiveness at the cost of additional inference costs.

In this work, we ask: **Can we apply the Matryoshka Principle to the Q-Nets produced by a Hypencoder?** To test this, we fine-tune an existing Hypencoder with an adaptation of the *Matryoshka* loss function [13] to produce Q-Nets of variable sizes (see Figure 1). We find that these smaller Q-Nets exhibit the expected trade-offs between efficiency and effectiveness, enabling a single model to be deployed across multiple operational points. Remarkably, we find that Q-Nets with approximately $7\times$ fewer active parameters provide comparable effectiveness as the full Q-Net in-domain. Keeping half the number of active parameters is required to consistently maintain out-of-domain effectiveness. We also show that reducing the Q-Net’s active parameter count reduces its computational cost.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGIR '26, Melbourne, VIC, Australia*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2599-9/2026/07

<https://doi.org/10.1145/3805712.3809980>

In summary, our contributions are as follows. (1) We propose the Matryoshka Hypencoder, a HyperNetwork capable of generating multiple sizes of query-specific relevance networks at once. (2) We show that we can adapt an existing Hypencoder into a Matryoshka Hypencoder through a lightweight fine-tuning process. (3) We show that the resulting model performs well on in-domain and out-of-domain evaluations, allowing for operational cost reductions while maintaining effectiveness. This work enables more practical deployment settings for the Hypencoder, and the reduced computational cost could allow the retrieval process to run on a CPU instead of GPU (further reducing deployment costs).

2 Background and Related Work

We now cover background on Hypencoders and MRL.

2.1 Hypencoders

The Hypencoder is a recent retrieval proposed by Killingback et al. [11] to addresses the limited expressiveness of linear similarity measurements such as inner products by representing the query as shallow neural network called the Q-Net, resulting in a type of models that balances the efficiency of bi-encoder models and the effectiveness of cross-encoder models

The Hypencoder model consists of a document encoder and a query encoder. The document encoder is a standard BERT-based encoder that produces a single [CLS] vector representation E_d for a document, the query encoder consists of a standard BERT-based encoder and an attention-based hyperhead forming a HyperNetwork [6] that is designed and trained to generate the parameters of the Q-Net. In Hypencoders a query q is passed through the query encoder producing embeddings $E_q \in R^{n \times h}$, where n is the number of tokens and h is the hidden dimension. The embeddings E_q are passed to the hyperhead generating the weights and biases of the Q-Net. This is implemented using a single-head attention mechanism and a feed-forward layer, producing the parameters of the Q-Net. For each of the l required parameter matrices of the Q-Net, the hyperhead computes Key and Value matrices and the hidden Representation:

$$K_i = E_q W_{K_i}, \quad V_i = E_q W_{V_i}, \quad H_i = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{h}}\right) V_i \quad (1)$$

where Q_i is a set of learnable query embeddings unique to the i -th parameter.

The Q-Net is represented as a shallow, multi-layer feed-forward network of a fixed architecture with dynamically generated weights. Killingback et al. [11] uses a 6-layer MLP with LayerNorm and residual connections. For a document embedding E_d , the relevance score s is computed by the forward pass of the Q-Net:

$$x_0 = E_d, \quad x_{i+1} = x_i + \text{LayerNorm}(\text{ReLU}(x_i W_i^q + b_i^q)) \quad (2)$$

where W_i^q and b_i^q are the parameters generated by the hyperhead for query q .

The results presented in the paper demonstrate that the Hypencoder architecture has outperformed strong bi-encoder baselines on in-domain tasks. More importantly, the performance gap widened substantially on the hard retrieval tasks, validating the strong expressive power of its Q-Nets.

2.2 Matryoshka Representation Learning (MRL)

While the Hypencoder improves the similarity measurement approach, it introduces a new rigidity, the fixed size of the Q-Net. However, we identify that Matryoshka Representation Learning (MRL) [13] could overcome this rigidity. MRL models produce a single, high-dimensional representation of the input, which can be effectively truncated to a lower-dimensional representation, much like a set of nested Matryoshka dolls (which inspired its name). The approach has also been applied to neural architectures themselves [14, 25]. These “2D” Matryoshka architectures extend the MRL concept along a second dimension: the *depth* of the encoder model. Truncating the model at an earlier layer reduces the number of activated parameters, thereby reducing computational cost.

Their approach trains a single Transformer encoder to produce effective embeddings from not only the final layer but also from various intermediate sub-layers. The total loss is a cumulative sum over both multiple dimensions (width) and multiple layers (depth). While 2DMSE focuses on producing truncatable embedding *vectors* by modifying the encoder’s training objective, our work explores a different generalisation of MRL. We focus on creating a truncatable *function* by applying the Matryoshka principle to the parameter space of the Hypencoder’s dynamically generated Q-Net.

MRL achieves this by modifying the loss functions. Instead of calculating the loss at the full dimension d the loss is calculated at multiple predefined smaller dimensions $\mathcal{M}$, or granularities (e.g. 128, 256,...etc) by simply truncating the representation vector to the smaller dimensions. The total loss is an aggregation of the losses over all the truncated representations. This forces the model to learn a representation that is well-structured and effective even at smaller dimensionalities. The MRL paper showed that using this training technique led to massive efficiency gains in tasks like classification and retrieval. The standard MRL loss function:

$$\min_{\{W^{(m)}\}_{m \in \mathcal{M}, \theta_F}} \frac{1}{N} \sum_{i \in [N]} \sum_{m \in \mathcal{M}} c_m \cdot \mathcal{L}\left(W^{(m)} \cdot F(x_i; \theta_F)_{1:m}; y_i\right) \quad (3)$$

where θ_F are the learnable parameters; $W^{(m)}$ is the classifier head for dimension m ; $\mathcal{M}$ is the set of Matryoshka dimensions; and c_m is the loss weighting coefficient for dimension m .

We adapt the principles of MRL from data representations to the relevance function itself. Instead of truncatable embedding vectors, we train a Matryoshka Hypencoder to generate a single set of parameters that can be truncated to form nested Q-Nets of varying widths, enabling flexible efficiency-effectiveness trade-offs.

3 The Matryoshka Hypencoder (MH)

We now cover the design of the Matryoshka Hypencoder.

3.1 Architecture

This work transforms the original MRL concept from generating a truncatable encoding vector into a truncatable function representation (the parameters of the Q-Net). The objective here is to train the hyperhead to generate a single large set of parameters Θ_q^{full} , for a wide Q-Net which can also be truncated into a subset of these parameters Θ_q^m , to form a valid and effective narrower Q-Net for dimension $m \in \mathcal{M}$.

Table 1: Evaluation dataset characteristics.

Dataset	In-Domain	# Queries	# Docs
MSMARCO Dev	✓	6,980	8.8M
DL'19	✓	43	8.8M
DL'20	✓	54	8.8M
TREC COVID (TC)	✗	50	171k
FiQA (FQA)	✗	648	58k
DBPedia (DB)	✗	400	4.6M
NFCorpus (NF)	✗	323	3.6k
Webis Touche (WT)	✗	49	383k

To achieve this, two key modifications are introduced to the baseline Hypencoder architecture. First, architectural changes are implemented to expose and truncate the generated parameters, and the original MarginMSE loss function is modified to become a Matryoshka multi-objective Margin MSE Loss function:¹

$$\mathcal{L} = \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \left([s_m(p^+; \Theta_q^{1:m}) - s_m(p^-; \Theta_q^{1:m})] - \Delta s_T(q) \right)^2 \quad (4)$$

During the training process the hyperhead generates the full set of parameters Θ_q^{full} , for the largest size Q-Net dimension m_{max} . The loss function iterates over every Matryoshka dimensions $m \in \mathcal{M}$ truncating the full set of parameters to a smaller parameter set Θ_q^m . A temporary Q-Net of a hidden layer width m is assembled using the truncated parameters. Then MarginMSE loss is calculated for this smaller Q-Net. Finally the losses of each dimension m are aggregated according to Eq. (4).

3.2 Practical Considerations

The training process of the original Hypencoder was very resource-intensive (e.g., 6 days on two A100 GPUs). This presents practical challenges for reproducibility, as well as environmental considerations. Instead of training the Matryoshka Hypencoder in a similar end-to-end fashion, we propose a lightweight training procedure to further fine-tune an existing Hypencoder model into a Matryoshka Hypencoder model. To further improve the training efficiency, we freeze the document and query transformer networks, allowing only the hyperhead of the query encoder to train. For fair comparisons, we include a version of the Hypencoder further fine-tuned this way, without the Matryoshka modifications.

4 Experiments

We conduct experiments to answer three research questions about the proposed Matryoshka Hypencoder. We first validate that our efficient fine-tuning procedure described in Section 3.2 is effective (without applying the Matryoshka Hypencoder architecture):

RQ1 Can a Hypencoder model, where the document and query encoders are frozen and only the hyperhead is trained, achieve performance comparable to the fully end-to-end trained model?

¹In a pilot study, we explored several variations combining the Matryoshka from Eq. (3) and standard MarginMSE. This version was the most reliable.

Table 2: In-domain performance comparison. MH d refers to our Matryoshka Hypencoder at a Q-Net hidden dimension of d . Hyp-Fr is our frozen-encoder model. † indicates a statistically significant difference ($p < 0.05$) compared to the Original Hypencoder baseline, and * shows significance compared to MH $d = 768$.

Model	# Q-Net Params	DL'19 nDCG@10	DL'20 nDCG@10	Dev MRR@10
<i>Our Matryoshka Hypencoder (MH)</i>				
MH $d = 768$	4,134,912	.734	.723	.380 [†]
MH $d = 512$	1,970,176	.738	.729	.381 [†]
MH $d = 256$	591,616	.741	.728	.379 [†]
MH $d = 128$	197,632	.734	.725	.375 ^{†*}
<i>Baselines</i>				
Hyp-Fr (Ours)	≈4.1M	.728	.729	.373 [†]
Original Hypencoder [11]	≈4.1M	.742	.730	.386
BM25	-	.498	.479	.188

Then, we run the same fine-tuning procedure under the Matryoshka Hypencoder architecture to answer the following questions:

- RQ2** Does the Matryoshka Principle apply to the space of Hyper-Networks under the Matryoshka Hypencoder architecture?
- RQ3** Is the Matryoshka Hypencoder model capable of generalising on out-of-domain retrieval tasks?
- RQ4** Are smaller Q-Nets produced by the Matryoshka Hypencoder progressively faster at inference?

4.1 Experimental Setup

For training all models, we use the pre-processed MS MARCO dataset from Killingback et al. [11], which includes passages scored by a cross-encoder teacher model for knowledge distillation. We set $\mathcal{M} = [128, 256, 512, 768]^2$ for the Matryoshka Hypencoder. Note that the original Hypencoder model used a fixed size of 768.

We evaluate the performance of the model on several standard benchmarks, summarised in Table 1. For in-domain evaluation, we use the MS MARCO Dev set [20] and the TREC Deep Learning (DL) 2019 and 2020 passage ranking tracks [3, 4]. To assess zero-shot out-of-domain (OOD) generalisation, we use a diverse suite of datasets from the BEIR benchmark [23], including TREC-COVID [24] (biomedical), NFCorpus [2] (nutrition), FiQA [17] (financial), DBPedia [7] (entity), and Touché [1] (argument retrieval). All evaluation datasets were accessed via the IR Datasets library [15]. This selection of training and evaluation data mirrors the setup in the original Hypencoder paper [11].

We measure the official task metric for each dataset: Mean Reciprocal Rank at rank 10 (MRR@10) for MS MARCO Dev, and Normalised Discounted Cumulative Gain at depth 10 (nDCG@10 [8]) for the other datasets. We compute statistical significance using a paired t-test and $p < 0.05$.

For context, we include the performance of the original Hypencoder model from [11] and the performance of PISA's [19] implementation of BM25 [22], using its default parameters.

²We also tried training smaller sizes (64 and 32), but the model failed to converge. This observation is in line with prior work that observed challenges when training Matryoshka with a small number of dimensions [25].

Table 3: Zero-shot performance (nDCG@10) on out-of-domain BEIR datasets. TC is TREC-COVID, FQA is FiQA, NF is NFCorpus, DB is DBPedia, and WT is Webis-Touché. † indicates a statistically significant difference ($p < 0.05$) compared to the Original Hypencoder baseline, and * shows significance compared to MH $d = 768$.

Model	TC	FQA	DB	NF	WT
<i>Our Matryoshka Hypencoder (MH)</i>					
MH $d = 768$	.685	.312	.414	.324	.272
MH $d = 512$	.684	.311	.414	.324	.271
MH $d = 256$	.678 [†]	.306 ^{†*}	.414	.324	.266
MH $d = 128$	.670 [†]	.304 ^{†*}	.412	.323	.259*
<i>Baselines</i>					
Hyp-Fr (Ours)	.699	.314	.419	.311 [†]	.250
Original Hypencoder [11]	.698	.314	.419	.324	.258
BM25	.610	.253	.303	.318	.332

5 Results and Discussion

This section answers the research questions posed in Section 4.

5.1 Frozen-Encoder Training (RQ1)

Our first research question explores whether a regular Hypencoder can be effectively fine-tuned with frozen transformers from an existing Hypencoder model (as described in Section 3.2). Tables 2 and 3 show the performance of this model (Hyp-Fr), compared with the results from the original Hypencoder. In 7 out of 8 datasets (across both in-domain and out-of-domain evaluations), we observe no statistically significant differences between the results from the original Hypencoder and this variant. This result demonstrates that the lightweight hyperhead training process is successful.

We answer **RQ1** in the affirmative: a Hypencoder model can be trained with frozen query and document encoders and achieve performance comparable to that of a fully end-to-end trained model.

5.2 Architectural Feasibility (RQ2)

Having established a viable training paradigm, we now evaluate our primary contribution: the Matryoshka Hypencoder.

Table 2 presents the in-domain performance of the Matryoshka Hypencoder model on the TREC DL 2019 and 2020 benchmarks and MSMARCO dev. All of the sizes ($M = [128, 256, 512, 768]$) exhibit strong retrieval effectiveness, and do not differ from the Original Hypencoder baseline when tested using near-complete relevant assessments (DL'19 and DL'20). However, it performs significantly worse than the Original Hypencoder baseline for the dev set, which contains incomplete relevance assessments. We also see no significant degradations compared to the full-size Matryoshka Hypencoder ($d = 768$) for $d = 512$ and $d = 256$, with only significant degradations for MS MARCO dev at $d = 128$.

Meanwhile, we observe substantial reductions in the number of active parameters: $d = 512$ needs less than half the number of parameters as the full-sized $d = 768$ model, while the reductions for the smaller sizes are more substantial (7× fewer parameters for $d = 256$ and 21× fewer parameters for $d = 128$). The non-linear

Table 4: Average document scoring throughput (documents scored per second). The speed up compared to the full-size Q-Net ($d = 768$) is given in parentheses.

Q-Net	Dev	TC	FQA	DB	NF	WT
$d = 768$	0.9M (1.0×)	0.9M (1.0×)	0.7M (1.0×)	0.9M (1.0×)	0.2M (1.0×)	0.9M (1.0×)
$d = 512$	1.4M (1.6×)	1.3M (1.5×)	0.9M (1.3×)	1.4M (1.6×)	0.2M (1.0×)	1.4M (1.6×)
$d = 256$	3.0M (3.4×)	2.4M (2.8×)	1.4M (2.0×)	3.0M (3.4×)	0.2M (1.1×)	2.7M (3.0×)
$d = 128$	5.6M (6.3×)	3.3M (3.9×)	1.8M (2.6×)	5.6M (6.3×)	0.2M (1.1×)	4.1M (4.6×)

relationship of the reductions is due to the fact that a reduction in the hidden size of the Q-Net's feed-forward network results in a roughly quadratic reduction in the number of parameters for the inner layers, as shown visually in Figure 1. This reduction is important because it allows more documents to be scored in parallel (when using a GPU) and potentially reduces the cost enough to run it on a CPU. We explore the efficiency of this reduction in RQ4.

These results answer **RQ2**: Matryoshka Hypencoder performs well in-domain: at $d = 256$ it yields no significant in-domain effectiveness reductions, while using 7× fewer active parameters.

5.3 Out-of-Domain Effectiveness (RQ3)

We next test how well the Matryoshka Hypencoder generalises to domains and tasks outside of the original training data (web question answering from MS MARCO). Table 3 presents results on the five BEIR datasets covered by the original Hypencoder paper. The in-domain findings hold across four of the five tasks (TREC-COVID, DBPedia, NFCorpus, and Webis Touche), with no statistically significant degradation compared to $d = 768$ for $d = 256$ and $d = 512$. Meanwhile, significant reductions were observed for FiQA at $d = 256$ and $d = 128$, suggesting that the lower sizes do lose some generalisation capacity. However, we do note that significant reductions compared to the original Hypencoder model were observed for both TREC-COVID and FiQA at $d = 256$ and $d = 128$. At $d = 512$, the number of active Q-Net parameters is reduced by half while maintaining comparable effectiveness across all five out-of-domain evaluation sets.

We answer **RQ3** in the affirmative: the Matryoshka Hypencoder generalises across domains, however it does lose some generalisation capacity as the size of the Q-Net is reduced.

5.4 Efficiency (RQ4)

Finally, we investigate the practical utility of our proposed architecture: Are smaller Q-Nets produced by the Matryoshka Hypencoder progressively faster at inference time? To answer this, we measured the average document scoring throughput (number of documents scored per second) of the Q-Net at different sizes on MS MARCO Dev and the five out-of-domain datasets. We only measure the Q-Net scoring time, ignoring the query encoding time, since that is roughly constant across all models and dominated by the transformer encoder part of the network. In all cases, we measure brute-force (exhaustive) retrieval over the collections on a NVIDIA RTX 5000 Ada GPU. We expect that the trends we observe in this setting will translate to throughput measurements when using other hardware (GPUs and CPUs) and when performing approximate search techniques, though we leave this for future work.

The results are presented in Table 4. There is a direct and negative correlation between the Q-Net’s hidden dimension and the throughput; as its size gets smaller, it becomes faster. This is expected, since a smaller Q-Net requires less computational cost to perform inference. The throughputs are most consistent on the two largest collections: MS MARCO (8.8M documents) and DBPedia (4.6M documents), since the larger samples make average per-document processing time more stable.³ They achieve a $6.3\times$ speed up at $d = 128$, which is well below the theoretical speed up based on the reduction in parameters ($20.9\times$). This discrepancy is likely due to memory bandwidth limitations. We suspect there may be ways to better optimise scoring, though we leave this for future work. In-domain, similar effectiveness was consistently achieved at $d = 256$, which results in a $3.4\times$ speed up. For out-of-domain, $d = 512$ consistently yielded similar effectiveness, which gives a speed up of $1.6\times$.

In summary, we answer **RQ4** affirmatively: smaller Q-Nets are progressively faster to execute. However, we do note that they fall short of the theoretical maximum based on parameter count, so future work could explore further optimising Q-Net inference.

6 Conclusion

In this work, we introduced the Matryoshka Hypencoder, a novel architecture that addresses the fixed-complexity limitation of the original Hypencoder. By adapting Matryoshka Representation Learning (MRL) principles to the parameter space of the Q-Net, we successfully trained a single hyperhead to generate nested, variable-width scoring functions. We demonstrated that a lightweight transfer learning approach, fine-tuning a pre-trained model with a multi-objective loss, is a viable and effective training strategy.

Our empirical results show that this architecture achieves an excellent trade-off between efficiency and parameter cost. On in-domain benchmarks, the full-size Matryoshka Q-Net usually performs on par with the original model, while a truncated Q-Net (256 hidden layer size) with $7\times$ fewer parameters achieves statistically comparable performance. Furthermore, we demonstrated that this favourable trade-off is robust, generalising successfully to a diverse suite of out-of-domain, zero-shot retrieval tasks. Crucially, we have shown that the reduction in active parameters translates to a massive gain in practical efficiency. The smaller Q-Nets are progressively faster at inference, with the 128-dimension Q-Net providing up to a $6.3\times$ reduction in query latency on large-scale corpora. This is due to the quadratic reduction in computational cost in the document scoring loop, which is the primary bottleneck in the retrieval process.

We hope that this generalisation of MRL to a function generator inspires more work in the query-specific relevance function space.

References

- [1] Alexander Bondarenko, Maik Fröbe, Meriem Beloucif, Lukas Gienapp, Yamen Ajjour, Alexander Panchenko, Chris Biemann, Benno Stein, Henning Wachsmuth, Martin Potthast, and Matthias Hagen. 2020. Overview of Touché 2020: Argument Retrieval. In *Working Notes of CLEF 2020 - Conference and Labs of the Evaluation Forum, Thessaloniki, Greece, September 22–25, 2020 (CEUR Workshop Proceedings, Vol. 2696)*, Linda Cappellato, Carsten Eickhoff, Nicola Ferro, and Aurélie Névoul (Eds.). CEUR-WS.org. https://ceur-ws.org/Vol-2696/paper_261.pdf
- [2] Vera Boteva, Demian Gholipour Ghalandari, Artem Sokolov, and Stefan Riezler. 2016. A Full-Text Learning to Rank Dataset for Medical Information Retrieval. In *Advances in Information Retrieval - 38th European Conference on IR Research, ECIR 2016, Padua, Italy, March 20–23, 2016. Proceedings (Lecture Notes in Computer Science, Vol. 9626)*, Nicola Ferro, Fabio Crestani, Marie-Francine Moens, Josiane Mothe, Fabrizio Silvestri, Giorgio Maria Di Nunzio, Claudia Hauff, and Gianmaria Silvello (Eds.). Springer, 716–722. doi:10.1007/978-3-319-30671-1_58
- [3] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, and Daniel Campos. 2021. Overview of the TREC 2020 deep learning track. *CoRR* abs/2102.07662 (2021). arXiv:2102.07662 <https://arxiv.org/abs/2102.07662>
- [4] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M. Voorhees. 2020. Overview of the TREC 2019 deep learning track. *CoRR* abs/2003.07820 (2020). arXiv:2003.07820 <https://arxiv.org/abs/2003.07820>
- [5] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning, ICLR 2020, 13–18 July 2020, Virtual Event (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 3887–3896. <http://proceedings.mlr.press/v119/guo20h.html>
- [6] David Ha, Andrew M. Dai, and Quoc V. Le. 2017. HyperNetworks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=rkpACe1lx>
- [7] Faegheh Hasibi, Fedor Nikolaev, Chenyan Xiong, Krisztian Balog, Svein Erik Bratsberg, Alexander Kotov, and Jamie Callan. 2017. DBpedia-Entity v2: A Test Collection for Entity Search. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval, Shinjuku, Tokyo, Japan, August 7–11, 2017*, Noriko Kando, Tetsuya Sakai, Hideo Joho, Hang Li, Arjen P. de Vries, and Ryan W. White (Eds.). ACM, 1265–1268. doi:10.1145/3077136.3080751
- [8] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Trans. Inf. Syst.* 20, 4 (2002), 422–446. doi:10.1145/582415.582418
- [9] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16–20, 2020*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, 6769–6781. doi:10.18653/V1/2020.EMNLP-MAIN.550
- [10] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25–30, 2020*, Jimmy X. Huang, Yi Chang, Xueqi Cheng, Jaap Kamps, Vanessa Murdock, Ji-Rong Wen, and Yiqun Liu (Eds.). ACM, 39–48. doi:10.1145/3397271.3401075
- [11] Julian Killingback, Hansi Zeng, and Hamed Zamani. 2025. Hypencoder: Hypernetworks for Information Retrieval. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2025, Padua, Italy, July 13–18, 2025*, Nicola Ferro, Maria Maistro, Gabriella Pasi, Omar Alonso, Andrew Trotman, and Suzan Verberne (Eds.). ACM, 2372–2383. doi:10.1145/3726302.3729983
- [12] Hrishikesh Kulkarni, Sean MacAvaney, Nazli Goharian, and Ophir Frieder. 2023. Lexically-Accelerated Dense Retrieval. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan, July 23–27, 2023*, Hsin-Hsi Chen, Wei-Jou (Edward) Duh, Hen-Hsen Huang, Makoto P. Kato, Josiane Mothe, and Barbara Poblete (Eds.). ACM, 152–162. doi:10.1145/3539618.3591715
- [13] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham M. Kakade, Prateek Jain, and Ali Farhadi. 2022. Matryoshka Representation Learning. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/c32319f4868da7613d78af9993100e42-Abstract-Conference.html
- [14] Xianming Li, Zongxi Li, Jing Li, Haoran Xie, and Qing Li. 2025. ESE: Espresso Sentence Embeddings. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=plgLA2YBLH>
- [15] Sean MacAvaney, Andrew Yates, Sergey Feldman, Doug Downey, Arman Cohan, and Nazli Goharian. 2021. Simplified Data Wrangling with ir_datasets. In *SIGIR '21: The 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, Virtual Event, Canada, July 11–15, 2021*, Fernando Diaz, Chirag Shah, Torsten Suel, Pablo Castells, Rosie Jones, and Tetsuya Sakai (Eds.). ACM, 2429–2436. doi:10.1145/3404835.3463254
- [16] Craig Macdonald, Nicola Tonellotto, and Zhili Shen. 2026. A Replicability Study of Joint Product Quantisation for Effective Space-Efficient Dense Retrieval. In *SIGIR '26: The 49th International ACM SIGIR Conference on Research and Development in*

³The smaller collections, especially NFCorpus (3.6k documents), had slightly less stable throughput measurements.

- Information Retrieval, Melbourne, Australia, July 20 - 24, 2026*. ACM. doi:0.1145/3805712.3808565
- [17] Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. 2018. WWW'18 Open Challenge: Financial Opinion Mining and Question Answering. In *Companion of the The Web Conference 2018 on The Web Conference 2018, WWW 2018, Lyon, France, April 23-27, 2018*, Pierre-Antoine Champin, Fabien Gandon, Mounia Lalmas, and Panagiotis G. Ipeirotis (Eds.). ACM, 1941–1942. doi:10.1145/3184558.3192301
 - [18] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
 - [19] Antonio Mallia, Michal Siedlaczek, Joel M. Mackenzie, and Torsten Suel. 2019. PISA: Performant Indexes and Search for Academia. In *Proceedings of the Open-Source IR Replicability Challenge co-located with 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, OSIRRC@SIGIR 2019, Paris, France, July 25, 2019 (CEUR Workshop Proceedings, Vol. 2409)*, Ryan Clancy, Nicola Ferro, Claudia Hauff, Jimmy Lin, Tetsuya Sakai, and Ze Zhong Wu (Eds.). CEUR-WS.org, 50–56. <https://ceur-ws.org/Vol-2409/docker08.pdf>
 - [20] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. In *Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016), Barcelona, Spain, December 9, 2016 (CEUR Workshop Proceedings)*, Tarek Richard Besold, Antoine Bordes, Artur S. d'Avila Garcez, and Greg Wayne (Eds.). CEUR-WS.org. https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf
 - [21] Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage Re-ranking with BERT. *CoRR* abs/1901.04085 (2019). arXiv:1901.04085 <http://arxiv.org/abs/1901.04085>
 - [22] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. 1994. Okapi at TREC-3. In *Proceedings of The Third Text REtrieval Conference, TREC 1994, Gaithersburg, Maryland, USA, November 2-4, 1994 (NIST Special Publication, Vol. 500-225)*, Donna K. Harman (Ed.). National Institute of Standards and Technology (NIST), 109–126. <http://trec.nist.gov/pubs/trec3/papers/city.ps.gz>
 - [23] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models. *CoRR* abs/2104.08663 (2021). arXiv:2104.08663 <https://arxiv.org/abs/2104.08663>
 - [24] Ellen M. Voorhees, Tasmeer Alam, Steven Bedrick, Dina Demner-Fushman, William R. Hersh, Kyle Lo, Kirk Roberts, Ian Soboroff, and Lucy Lu Wang. 2020. TREC-COVID: constructing a pandemic information retrieval test collection. *SIGIR Forum* 54, 1 (2020), 1:1–1:12. doi:10.1145/3451964.3451965
 - [25] Shuai Wang, Shengyao Zhuang, Bevan Koopman, and Guido Zucco. 2025. 2D Matryoshka Training for Information Retrieval. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2025, Padua, Italy, July 13-18, 2025*, Nicola Ferro, Maria Maistro, Gabriella Pasi, Omar Alonso, Andrew Trotman, and Suzan Verberne (Eds.). ACM, 3125–3134. doi:10.1145/3726302.3730330
 - [26] Orion Weller, Michael Boratko, Iftekhar Naim, and Jinhyuk Lee. 2025. On the Theoretical Limitations of Embedding-Based Retrieval. *CoRR* abs/2508.21038 (2025). arXiv:2508.21038 doi:10.48550/ARXIV.2508.21038